

1 >>>

数字逻辑概论

引言

随着现代电子技术的发展,人们正处于一个信息时代。每天都要通过电视、广播、通信、互联网等多种媒体获取大量的信息。而现代信息的存储、处理和传输越来越趋于数字化。在人们的日常生活中,常用的计算机、电视机、音响系统、视频记录设备、远距离通信等电子设备或电子系统中,无一不采用数字电路或数字系统。因此,数字电子技术的应用越来越广泛。

本章首先介绍数字技术的发展及应用,数字集成电路的分类及特点、模拟信号和数字信号以及数字信号的描述方法,然后讨论数制、二进制的算术运算、二进制代码和数字逻辑的基本运算。

1.1 数字信号与数字电路

1.1.1 数字技术的发展及其应用

电子技术是 20 世纪以来发展最迅速、应用最广泛的技术,已经渗透到人类生活的各个方面。特别是数字电子技术,取得了令人瞩目的进步。

电子技术的发展是以电子器件的发展为基础的。20 世纪初直至中叶,主要使用真空管,也称电子管。随着固体微电子学的进步,第一只晶体管于 1947 年问世,开创了电子技术的新领域。随后,在 20 世纪 60 年代初,模拟和数字集成电路相继上市。20 世纪 70 年代微处理器的问世,使电子器件及其应用出现了崭新的局面。20 世纪 80 年代末,微处理器每个芯片的晶体管数目突破百万大关。到 20 世纪 90 年代末,可以制造包含千万个晶体管的芯片。当前的制造技术可以在芯片上集成几十亿个晶体管。在过去的 40 年间,集成电路的集成度和性能以惊人的速度发展着,印证了摩尔定理,即单个芯片上集成的晶体管数量每 18 个月就翻一番。以集成电路为核心的电子信息产业已超过了汽车、石油和钢铁工业成为第一大产业。

数字技术应用的典型代表是电子计算机,它是伴随着电子技术的发展而发展的,经历了电子管、晶体管、集成电路和超大规模集成电路四个发展阶段,其体积越来越小,功能越来越强,价格越来越低,应用范围越来越广,目前正朝着智能化方向发展。计算机技术的影响已遍及人类经济生活的各个领域,掀起了一场“数字革命”。数字技术被广泛地应用于广播、电视、通信、医学诊断、测量、控制、文化娱乐以及家庭生活等方面。由于数字信息具有便于存储、处理和传输的特点,使得许多传统使用模拟技术的领域转而运用数字技术。

音频信息存储 20 世纪 90 年代以前,声音的存储主要以模拟信号的方式将声波记录在唱片或磁带上,而两者的携带和保存都不方便。现在是将声音转换成数字信号存储在 CD (Compact Disc) 上。存储音乐的 CD 通常使用的取样频率为 44.1 kHz,量化位数为 16 位,同时有两股声音被录制(分别流向立体声系统的两个扬声器),而且可以存储长达 74 min 的音乐。这样一张 CD 盘上储存的数字信息总量超过 700 MB。CD 盘可用于存储不同格式的数据,最常见的格式是音频数据和计算机数据。

视频信息存储 DVD (Digital Video Disk 或 Digital Versatile Disk) 普及之前,早期的视频信息存储主要以记录模拟信号的录像带为主,而录像带不便于保存和传输。DVD 与 CD 外观类似,但在数据存储技术、数据容量及功能等方面都有本质的区别。CD 存储无数据压缩的音频信息,而 DVD 采用 MPEG2 的压缩技术,以数字格式存储音频和视频信息。DVD 可以分为单面单层、单面双层、双面单层和双面双层四种物理结构,其数据容量非常大,画质和音质好。仅单面单层、直径 12 cm 的光盘就能存储 4.7 GB 数据,影片播放时间可达 133 min。双面双层存储的数据可高达 17 GB 之多,影片播放时间可达 8 h 以上。因此,DVD 已成为家庭影院的重要组成部分。

MPEG (Moving Picture Experts Group) 是世界数字视频和音频压缩比的标准化组织制定的,用于多媒体运动图像和伴音的数据压缩编码的国际标准。MPEG 标准主要包括 MPEG1、MPEG2、MPEG4、MPEG7 和 MPEG21 等,以适用于不同带宽和数字影像质量的要求。MPEG 的压缩比,最高可达 200:1,而且在提供高压缩比的同时,对数据的损失很小。MPEG 标准改变了以模拟方式记录声音和影像的传统方法,令视听传播进入到数字化时代。

照相机 传统的模拟相机是用卤化银感光胶片记录影像,胶片成像过程需要严格的加工工艺和技术,而且胶片不便于保存和传输。数码相机是将影像的光信号转换为数字信号,以像素阵列的形式进行存储。存储的信息包括色彩、光强度和位置等。例如 1024×768 的像素阵列中,每个像素的红、绿、蓝三原色均是 8 位,则该阵列的数据位数超过 1800 万。如果用 JPEG 图形格式进行压缩处理,压缩比率采用 20:1,处理后的数据量只为原来的 5%,便于进行网络的远距离传输。如今,数码相机已完全取代模拟胶片相机。

JPEG (Joint Photographic Experts Group) 是国际标准化组织 (International Standard Organization, ISO) 和国际电报电话咨询委员会 (Consultative Committee for International Telegraph and Telephone, CCITT) 联合制定的静止图像压缩编码标准。JPEG 有多种压缩级别,压缩比率通常在 10:1 到 40:1 之间,是目前较流行的静止图像压缩文件格式。

交通灯控制系统 交通灯于 1920 年问世。早期的红黄绿灯是用机电定时器控制的。后来

使用由电子线路和继电器构成的控制器,根据道路上传感器检测的信号进行控制,可靠性低,故障率高。现在的交通灯由计算机控制,可以将监测系统检测到的车辆流量信息送到系统计算机,经计算后进行合理的时间分配。如某路口东西方向堵塞,则将该路口东西方向的绿灯自动延时,并将附近区域东西方向的红灯也自动地延时,堵塞解除后,信号灯恢复正常状态。

随着微电子技术的发展,将会有更多的数字电子产品陆续问世。数字技术的发展、计算机的应用正在改变着人类的生产方式、生活方式及思维方式,它使得工业自动化、农业现代化、办公自动化和通信网络化成为现实。但是,无论数字技术如何发展,也不能代替模拟技术。自然界中绝大多数物理量都是模拟量,数字技术不能直接接受模拟信号进行处理,也无法将处理后的数字信号直接送到外部物理世界。因此,模拟技术在电子系统中是不可缺少的。由于模拟技术难度远高于数字技术,其发展自然较慢。实际电子系统一般是模拟电路和数字电路的结合,在发展数字技术的同时,也应重视模拟技术的发展。

1.1.2 数字集成电路的分类及特点

电子电路按功能分为模拟电路和数字电路。根据电路的结构特点及其对输入信号的响应规则的不同,数字电路可分为**组合逻辑电路**和**时序逻辑电路**。数字电路中的电子器件,如二极管、三极管(FET、BJT)处于开关状态,时而导通,时而截止,构成电子开关。这些电子开关是组成逻辑门电路的基本器件。逻辑门电路又是数字电路的基本单元。如果将这些门电路集成在一片半导体芯片上就构成数字集成电路。

1. 数字集成电路的分类

数字电路的发展历史与模拟电路一样,经历了由电子管、晶体管等半导体分立器件到集成电路的发展过程。由于集成电路的发展非常迅速,很快占有主导地位,因此,数字电路主流形式是数字集成电路。从20世纪60年代开始,数字集成器件以双极型工艺制成了小规模逻辑器件,随后发展到中规模;20世纪70年代末,微处理器的出现,使数字集成电路的性能发生了质的飞跃;从20世纪80年代中期开始,专用集成电路(Application Specific Integrated Circuit, ASIC)制作技术已趋成熟,标志着数字集成电路发展到了新的阶段;20世纪90年代中期,片上系统(System on Chip, SoC)设计技术可以将复杂的电子系统全部集成在单一芯片上,使集成电路设计向集成系统设计转变,预示着集成电路出现了从量变到质变的突破。

ASIC是根据用户特定要求和电子系统的特定需要而设计制造的专用集成电路。与通用集成电路相比具有体积小、重量轻、功耗低、速度高、成本低、保密性强的优点。ASIC芯片的制作可以采用全定制或半定制的方法。全定制适用于生产批量的成熟产品,由半导体生产厂家制造。对于生产批量小或研究试制阶段的产品,可以采用半定制方法。目前最为流行的半定制方法是用复杂可编程逻辑器件(Complex Programmable Logic Device, CPLD)和现场可编程门阵列(Field Programmable Gate Array, FPGA)来进行ASIC设计。用户通过软件编程,将自己设计的数字系统制作在厂家生产的CPLD或FPGA芯片上,便得到所需的系统级芯片。

SoC是将电子系统中所有不同的功能块集成在一个芯片中,称为片上系统,如手机芯片、数

字电视芯片、DVD 芯片等。SoC 芯片的规模远大于普通的 ASIC,其设计方法进一步分工细化,出现了 IP(Intellectual Property)设计和 SoC 系统设计。IP 内核模块是一种已经过验证的、可重利用的、具有某种确定功能的模块。将数字电路中常用的但比较复杂的功能块,如微处理器、微控制器、滤波器、嵌入式存储器、数字信号处理器等设计成可修改参数的模块,让用户直接调用这些模块,这样可以大大减轻工程师的负担,避免重复劳动。随着 CPLD/FPGA 的规模越来越大,设计越来越复杂,使用 IP 核是一个发展趋势。

衡量集成电路有两个主要的参数:集成度与特征尺寸。集成度是指每一芯片所包含的门的个数。特征尺寸是指集成电路中半导体器件加工的最小线条宽度。集成度与特征尺寸是相关的。当集成电路芯片的面积一定时,集成度越高,特征尺寸就越小。所以,特征尺寸也成为衡量集成电路设计和制造技术水平高低的重要指标。

在过去的几十年中,以硅为主要加工材料的微电子制造工艺从开始的微米级加工水平,经历亚微米级($0.8 \sim 0.35 \mu\text{m}$)、深亚微米级($0.25 \mu\text{m}$ 以下)技术的发展,到现在的纳米级($0.05 \mu\text{m}$ 以下)技术,集成电路芯片集成度越来越高,成本越来越低。

从集成度来说,数字集成电路可分为小规模(Small Scale Integration, SSI)、中规模(Medium Scale Integration, MSI)、大规模(Large Scale Integration, LSI)、超大规模(Very Large Scale Integration, VLSI)和甚大规模(Ultra Large Scale Integration, ULSI)五类。表 1.1.1 列出了数字集成电路的集成度分类。

表 1.1.1 数字集成电路的集成度分类

分类	门的个数	典型集成电路
小规模	最少 1 个	逻辑门、触发器
中规模	$12 \sim 99$	计数器、加法器
大规模	$100 \sim 9999$	小型存储器、门阵列
超大规模	$10\,000 \sim 99\,999$	大型存储器、微处理器
甚大规模	10^6 以上	可编程逻辑器件、多功能专用集成电路

逻辑门是数字集成电路的主要单元电路,按照结构和工艺分为双极型、MOS 型和双极-MOS 型。三极管-三极管(Transistor-Transistor Logic, TTL)逻辑门电路问世较早,其工艺经过不断改进,曾广泛应用于中小规模集成电路设计中,目前的使用范围大大缩小。随着金属-氧化物-半导体(MOS)工艺特别是 CMOS(Complementary Metal-Oxide-Semiconductor)工艺的发展,大大提高了电路的集成度和工作速度,且明显降低了功耗,因此 TTL 的主导地位已被 CMOS 器件所取代。

2. 数字集成电路的特点

与模拟电路相比,数字电路主要有下列优点。

(1) 稳定性高,抗干扰能力强

一般而言,对于一个给定的输入信号,如果噪声信号不使其超过阈值,数字电路的输出状态不变。因此数字电路的工作可靠,稳定性好,抗干扰能力强。而模拟电路的输出则随着外界温度和电源电压的变化,以及器件的老化等因素而发生变化。

(2) 易于设计

数字电路又称为数字逻辑电路,它主要是对用0和1表示的数字信号进行逻辑运算和处理,不需要复杂的数学知识,广泛使用的数学工具是逻辑代数。数字电路只要能够可靠地区分0和1两种状态就可以正常工作。因此,数字电路的分析与设计相对较容易。

(3) 便于集成,成本低廉

数字电路便于集成化生产,成本低廉,体积小,通用性强,容易制造,进而使集成芯片广泛应用于数字电路。

(4) 可编程性

现代数字系统的设计,大多采用可编程逻辑器件,即厂家生产的一种半成品芯片。用户根据需要用硬件描述语言(Hardware Description Language, HDL)在计算机上完成电路设计和仿真,并写入芯片,这为用户研制开发产品带来了极大的方便和灵活性。

(5) 高速度,低功耗

随着集成电路工艺的发展,数字器件的工作速度越来越高,而功耗越来越低。集成电路中单管的开关速度可以做到小于 10^{-11} s。整体器件中,信号从输入到输出的传输时间小于 10^{-9} s。百万门以上超大规模集成芯片的功耗,可以低于毫瓦级。

(6) 便于存储、传输和处理

数字信号由0和1编码组成,可以用数字存储器对其进行存储、传输。将数字系统与计算机连接,便可利用计算机对数字信号进行控制和。

由于具有这些优点,可以肯定,数字电路在众多领域取代模拟电路的趋势将会继续发展下去。

3. 数字电路的分析、设计与测试

(1) 数字电路的分析方法

数字电路处理的是数字信号,电路中的半导体器件工作在开关状态,如晶体管工作在饱和区或截止区,所以不能采用模拟电路使用的小信号模型等方法进行分析。数字电路又称为逻辑电路,在电路结构、功能和特点等方面均不同于模拟电路,因而,其分析方法与模拟电路完全不同。数字电路的主要研究对象是电路的输出与输入之间的逻辑关系,所采用的分析工具是逻辑代数,表达电路输出与输入的关系主要用真值表、功能表、逻辑表达式或波形图。

随着计算机技术的发展,借助电子设计自动化(Electronic Design Automation, EDA)工具,可以更直观、更快捷、更全面地对电路进行分析。EDA软件可以用于模拟电路、数字电路或模数混合电路进行仿真分析。用EDA软件进行电路的功能仿真时,可以显示逻辑仿真的波形结果,以检查逻辑错误。如果进行时序仿真时,可以设置器件及连线的延迟时间,以便检测电路中存在的冒险竞争、时序错误等问题。

(2) 数字电路的设计方法

数字电路的设计是从给定的逻辑功能要求出发,确定输入、输出变量,选择适当的逻辑器件,设计出符合要求的逻辑电路。设计过程一般有方案的提出、验证和修改三个阶段。设计方式分为传统的设计方式和基于 EDA 软件的设计方式。传统的硬件电路设计全过程都是由人工完成,硬件电路的验证和调试是在电路构成后进行的,电路存在的问题只能在验证后发现。如果存在的问题较大,有可能重新设计电路,因而设计周期长,资源浪费大,不能满足大规模集成电路设计的要求。基于 EDA 软件的设计方式是借助于计算机来快速准确地完成电路的设计。设计者提出方案后,利用计算机进行逻辑分析、性能分析、时序测试,如果发现错误或方案不理想,可以重复上述过程直至得到满意的电路,然后进行硬件电路的实现。这种方法提高了设计质量,缩短了设计周期,节省了设计费用,提高了产品的竞争力。因此 EDA 软件已成为设计人员不可或缺的有力工具。

EDA 软件的种类较多,大多数软件包含以下主要工具。

原理图输入 设计者可以如同在纸上画电路一样,将逻辑电路图输入到计算机,软件自动检查电路的接线、电源及地线的连接、信号的连接等。

HDL 文本输入 硬件描述语言是用文本的形式描述硬件电路的功能、信号连接关系以及时序关系。它虽然没有图形输入那么直观,但功能更强,可以进行大规模、多个芯片的数字系统的设计。常用的 HDL 语言有 VHDL (Very High Speed Integrated Circuit) 和 Verilog HDL 等。

测试平台 当逻辑电路的设计输入到计算机后,需要在测试平台上编写或绘制激励信号,以便测试验证电路逻辑功能或时序关系的正确性。

仿真和综合工具 仿真工具包括对电路的功能仿真和时序仿真。功能仿真用于验证电路的功能和逻辑关系是否正确。时序仿真考虑门及连线的延时,验证系统内部工作过程及输入输出的时序关系是否满足设计要求。

综合工具 将 HDL 描述的电路的逻辑关系,转换为门和触发器等元件及其相互连接的电路形式。

(3) 数字电路的测试技术

数字电路在正确设计和安装后,必须经过严格的测试方可使用。测试中小规模数字电路时通常使用下列基本仪器设备。

数字电压表 用来测量电路中各点的电压,并观察其测试结果是否与理论分析一致。

数字示波器 常用来观察电路各点的波形。一个复杂的数字系统,在系统时钟信号的激励下,有关逻辑关系可以从波形图中得到验证。逻辑分析仪是一种专用示波器,例如,它可以同时显示 8~32 位的数字波形,非常有利于对整体电路各部分之间的逻辑关系进行分析。

集成电路的复杂程度越高,测试越困难,对于系统芯片 SoC 的测试成本几乎占芯片成本的一半。大规模集成电路的测试系统由测试硬件和测试软件组成。测试硬件除计算机外,还有测试仪和控制测试仪的处理器。测试软件包括操作系统、开发系统、运行系统和报告系统等。集成电路测试面临的最大挑战是如何降低测试成本。

1.1.3 模拟信号和数字信号

1. 模拟信号

人们在自然界感知的许多物理量中,有一些物理量如速度、压力、温度、声音、重量以及位置等具有一个共同的特点,即它们在时间上是连续变化的,幅值上也是连续取值的。这种连续变化的物理量称为模拟量,表示模拟量的信号称为模拟信号,处理模拟信号的电子电路称为模拟电路。在工程技术上,为了便于处理和分析,通常用传感器将模拟量转换为与之成比例的电压或电流信号,然后再送到电子系统中进一步处理。图 1.1.1(a)中给出了由热电偶得到的一个模拟电压信号波形。

2. 数字信号

与模拟量相对应的另一类物理量称为数字量。它们是在一系列离散的时刻取值,数值的大小和每次的增减都是量化单位的整数倍,即它们是一系列时间离散、数值也离散的信号。表示数字量的信号称为数字信号。将工作于数字信号下的电子电路称为数字电路。例如用温度计测量某一天内的温度变化,仅在整点时刻读取数据,并且对数据进行量化,若某次温度计的读数为 30.35°C ,取 1°C 作为量化单位,则温度记录值为 30°C 。这样,一天内的温度记录在时间上和数值上都是离散的,温度是以 1°C 为单位增加或减少。显然,用数字信号也可以表示其他各种物理量的大小,只是存在着一定的误差,误差取决于量化单位大小的选择。

随着计算机的广泛应用,绝大多数电子系统都采用计算机对信号进行处理。由于计算机无法直接处理模拟信号,所以需要将模拟信号转换为数字信号。

3. 模拟量的数字表示

图 1.1.1 所示为转换过程中的各种波形图,图 1.1.1(a)所示是模拟电压信号。首先对模拟信号取样。图 1.1.1(b)所示为模拟信号通过取样电路后,变成时间离散、幅值连续的取样信号, $t_0, t_1, t_2, \dots$ 为取样时间点。这里,幅值连续是指各取样点的幅值没有量化,仍然与对应的模拟信号的幅值相同,如图 1.1.1(a)和图 1.1.1(b)中 t_1 处的幅值均为模拟量 0.915V 。然后对取样信号进行量化,即数字化。选取一个量化单位,将取样信号除以量化单位并取整数结果,得到时间离散、数值也离散的数字量。最后对得到的数字量进行编码,生成用 0 和 1 表示的数字信号,如图 1.1.1(c)所示。图中以 0.1V 作为量化单位,对 t_1 处的幅值 0.915V 进行量化,量化后数值为 9。该值用 8 位二进制数表示为 00001001。如果取样点足够多,量化单位足够小,数字信号可以较真实地反映模拟信号。关于模数和数模转换的详细讨论见本书第 10 章的内容。

1.1.4 数字信号的描述方法

模拟信号可以用数学表达式或波形图等表示。数字信号则用 0、1 两种值表示,即二值数字逻辑(Binary Digital Logic);或高、低电平组成的数字波形,即逻辑电平(Logic Level)表示。

1. 二值数字逻辑和逻辑电平

在数字电路中,可以用 0 和 1 组成的二进制数表示数量的大小,也可以用 0 和 1 表示两种不

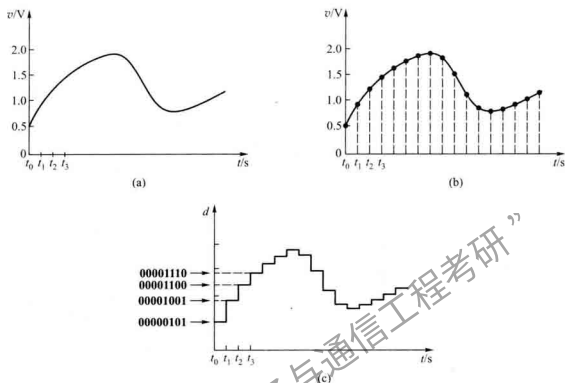


图 1.1.1 模拟量的数字表示
(a) 模拟电压信号 (b) 取样信号 (c) 数字信号

同的逻辑状态。当表示数量时,两个二进制数可以进行数值运算,常称为算术运算,将在 1.3 节介绍。当用 **0** 和 **1** 描述客观世界存在的彼此相互关联又相互对立的事物时,例如,是与非,真与假,开与关,低与高,通与断等等,这里的 **0** 和 **1** 不是数值,而是逻辑 **0** 和逻辑 **1**。这种只有两种对立逻辑状态的逻辑关系称为**二值数字逻辑**或简称**数字逻辑**。

在电路中,可以很方便地用电子器件的开关来实现二值数字逻辑。也就是以高、低电平分别表示逻辑 **1** 和 **0** 两种状态。在分析实际数字电路时,考虑的是信号之间的逻辑关系,只要能区别出表示逻辑状态的高、低电平,就可以忽略高、低电平的具体数值。表 1.1.2 所示为一种类型 CMOS 器件的电压范围与逻辑电平之间的关系。当信号电压在 $3.5 \sim 5\text{ V}$ 范围内,都表示高电平;在 $0 \sim 1.5\text{ V}$ 范围内,都表示低电平。这些表示数字电压的高、低电平通常称为**逻辑电平**。应当注意,逻辑电平不是物理量,而是物理量的相对表示。

表 1.1.2 电压范围与逻辑电平的关系

电压	二值逻辑	电平
$3.5 \sim 5\text{ V}$	1	H(高电平)
$0 \sim 1.5\text{ V}$	0	L(低电平)

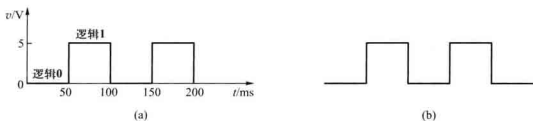


图 1.1.2 数字波形

(a) 标明时间及幅值的数字波形 (b) 数字波形的常规表示

图 1.1.2 所示为用逻辑电平描述的数字波形,用逻辑 0 表示低电平,逻辑 1 表示高电平。其中图 1.1.2(a) 所示波形标出时间及幅值。通常在分析一个数字系统时,由于电路采用相同的逻辑电平标准,一般可以不标出高、低电平的电压值。时间轴也可以不标,如图 1.1.2(b) 所示。

2. 数字波形

(1) 数字波形的两种类型

数字波形是逻辑电平对时间的图形表示。数字信号有两种传输波形,一种是非归零型,另一种是归零型。在图 1.1.3 中,一定的时间间隔 T ,称为 1 位(1bit),或者一拍。如果在一个时间拍内用高电平代表 1,低电平代表 0,称为非归零型,如图 1.1.3(a) 所示。如果在一个时间拍内有脉冲代表 1,无脉冲代表 0,称为归零型,如图 1.1.3(b) 所示。两者的区别在于高电平的表示方法不同,在一个时间拍内,非归零型信号持续为高电平,而归零型信号的高电平持续一段时间后归零。只有作为时序控制信号使用的时钟脉冲是归零型,除此之外的大多数数字信号基本都是非归零型,非归零型信号使用较为广泛。

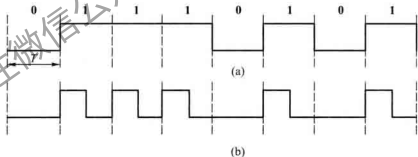


图 1.1.3 数字信号的传输波形

(a) 非归零型信号 (b) 归零型信号

数字信号只有两种取值,故称为二值信号,数字波形又称为二值波形图。非归零型信号的每位数据占用一拍时间。每秒钟所传输数据的位数称为数据率或比特率(Bit Rate)。

例 1.1.1 某通信系统每秒钟传输 1544000 位(1.544 兆位)数据,求每位数据的时间。

解: 按题意,每位数据的时间为

$$\left(\frac{1.544 \times 10^6}{1 \text{ s}}\right)^{-1} = 647.67 \times 10^{-9} \text{ s} \approx 648 \text{ ns}$$

(2) 周期性和非周期性

与模拟波形相同,数字波形亦有周期性与非周期性之分。图 1.1.4(a)、(b)所示分别为周期性与非周期性数字波形举例。

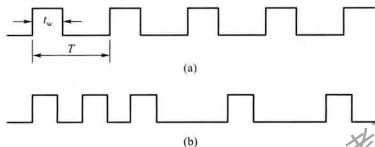


图 1.1.4 数字波形

(a) 周期性数字波形 (b) 非周期性数字波形

周期性数字波形常用周期 T 和频率 f 来描述。脉冲波形的脉冲宽度用 t_w 表示,它表示脉冲的作用时间。另一个重要参数是占空比 q ,它表示脉冲宽度 t_w 占整个周期 T 的百分数,常用下式来表示

$$q(\%) = \frac{t_w}{T} \times 100\% \quad (1.1.1)$$

当占空比为 50% 时,称此时的矩形脉冲为方波,即 0 和 1 交替出现并持续占有相同的时间。

例 1.1.2 设周期性数字波形的高电平持续 6 ms,低电平持续 10 ms,求占空比 q 。

解: 因数字波形的脉冲宽度 $t_w = 6 \text{ ms}$,周期 $T = 6 \text{ ms} + 10 \text{ ms} = 16 \text{ ms}$ 。

$$q = \frac{6 \text{ ms}}{16 \text{ ms}} \times 100\% = 37.5\%$$

(3) 实际数字信号波形

在实际的数字系统中,数字信号并没有那么理想。当矩形脉冲从低电平跳变到高电平,或从高电平跳到低电平时,边沿没有那么陡峭,而要经历一个过渡过程,分别用上升时间 t_r 和下降时间 t_f 描述,如图 1.1.5 所示。若脉冲幅值为 V_m ,将矩形脉冲从 10% V_m 到 90% V_m 时所经历的时间

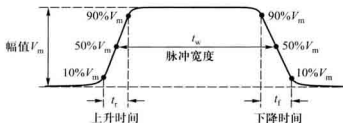


图 1.1.5 非理想脉冲波形

间称为上升时间 t_r 。下降时间则相反,将矩形脉冲从 $90\% V_m$ 下降到 $10\% V_m$ 时所经历的时间称为下降时间 t_f 。将脉冲上升沿的 $50\% V_m$ 到下降沿的 $50\% V_m$ 两个时间点所跨越的时间称为脉冲宽度 t_w ,对于不同类型的器件和电路,其上升和下降时间各不相同。一般数字信号上升和下降时间的典型值约为几纳秒(ns)。

例 1.1.3 试绘出一幅值为 5 V 的脉冲波形,设它的占空比为 50% ,脉冲宽度为 100 ns ,上升时间为 10 ns ,而下降时间为 20 ns 。

解: 根据题意,所绘出的脉冲波形如图 1.1.6 所示。

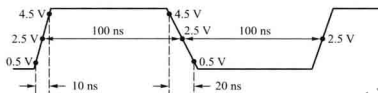


图 1.1.6 例 1.1.3 的波形图

本书所用的数字波形大多数将画成理想波形。实际上,每一波形均有上升时间 t_r 和下降时间 t_f 。当我们关注各信号之间的逻辑关系时,不必在每一波形上表示出上升和下降时间。只有当研究数字电路的翻转特性时,才绘出各信号的上升时间和下降时间。

(4) 波形图、时序图或定时图

数字波形是表达数字电路动态特性的有效工具之一。将数字电路输入变量的每一种取值与相应的输出值按照时间顺序依次排列得到的图形,称为波形图(Waveform)。

在时序电路中,电路的状态和输出对时钟脉冲序列和输入信号响应的波形图称为时序图或定时图(Timing Diagram)。时序图用来表示多个输入信号的先后顺序,以及输出如何对输入信号产生响应的过程。通常时序图侧重描述电路逻辑功能,而定时图侧重各个信号的先后顺序以及时间量。为便于比较,图 1.1.7 所示为 D 触发器的波形图。图中 CP 为时钟控制输入, D 为数据输入, Q 为输出。在图 1.1.7(b)中为保证电路稳定工作, D 输入端的信号比 CP 上升沿提前至少

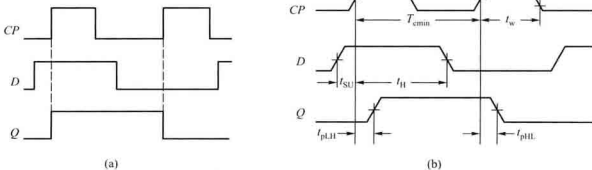


图 1.1.7 D 触发器的波形图

(a) 时序图 (b) 定时图

t_{st} (建立时间), 并且要求其在 CP 上升沿来之后保持至少 t_{H} (保持时间), 其他时间参数在后续章节介绍。通常数字集成电路, 例如存储器和时序逻辑器件等均附有时序图, 以便于进行数字系统的分析、设计和应用。

复习思考题

- 1.1.1 衡量集成电路有哪两个主要的参数?
- 1.1.2 数字电路从集成度来看可分为几大类?
- 1.1.3 按照结构和工艺分类数字电路有哪两种主要系列?
- 1.1.4 数字电路有哪些优点?
- 1.1.5 为什么数字逻辑称为二值数字逻辑?
- 1.1.6 实际数字波形和理想数字波形有什么不同?

1.2 数制

人们在日常生活中经常遇到计数问题, 并且习惯于用十进制数。而在数字系统, 例如计算机中, 通常采用二进制数, 有时也采用十六进制数或八进制数。这种多位数码的构成方式以及从低位到高位进位的规则称为数制。

1.2.1 十进制

众所周知, 中国的珠算盘是一个十进制计数器。任何一个数都可以用 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 这 10 个数码中的一个或几个, 按一定的规律排列起来表示, 其计数规律是“逢十进一”即 $9+1=10$, 其中左边的“1”为十位数, 右边的“0”为个位数, 也就是 $10=1 \times 10^1 + 0 \times 10^0$ 。所谓十进制就是以 10 为基数的计数体制。

这样, 每一数码处于不同的位置时, 它所代表的数值是不同的。例如, 十进制数 4587.29 可以表示为

$$4587.29 = 4 \times 10^3 + 5 \times 10^2 + 8 \times 10^1 + 7 \times 10^0 + 2 \times 10^{-1} + 9 \times 10^{-2}$$

式中 10^3 、 10^2 、 10^1 和 10^0 分别为千位、百位、十位和个位数码的权, 而小数点以右数码的权值是 10 的负幂。这与珠算盘横梁上所标示的个、十、百、千的位权是相同的。

一般地说, 任意十进制数可表示为

$$(N)_D^{①} = \sum_{i=-\infty}^{\infty} K_i \times 10^i \quad (1.2.1)$$

式中 K_i 为基数“10”的第 i 次幂的系数, 它可以是 0~9 中任何一个数字。

① 下标 D (Decimal) 表示十进制。

如果将式(1.2.1)中的10用字母 R 来代替,就可以得到任意进制数的表达式

$$(N)_R = \sum_{i=-\infty}^n K_i \times R^i \quad (1.2.2)$$

式中 K_i 是第 i 次幂的系数,根据基数 R 的不同,它的取值为0到 $R-1$ 个不同的数码。例如对于十进制数, R 为10,所以 K_i 的取值为0~9共10个数码。

数字电路不便于存储或处理十进制数。因为构成数字电路的基本思路是把电路的状态与数码对应起来。二进制只有0和1两个数码,分别用数字电路的两种状态来表示。而十进制的10个数码要求电路有10个完全不同的状态,这样使得电路很复杂,因此在数字电路中不直接处理十进制数。

1.2.2 二进制

1. 二进制的表示方法

二进制数中,只有0和1两个数码,并且计数规律是“逢二进一”,即 $1+1=10$ (读为“壹零”)。必须注意,这里的“10”与十进制数的“10”是完全不同的,它并不代表数“拾”。左边的“1”表示 2^1 的系数,右边的“0”表示 2^0 的系数,也就是 $10=1 \times 2^1 + 0 \times 2^0$ 。因此,所谓二进制就是以2为基数的计数体制。

根据式(1.2.2),任意二进制数可表示为

$$(N)_{B_2} = \sum_{i=-\infty}^n K_i \times 2^i \quad (1.2.3)$$

式中 K_i 为基数“2”的第 i 次幂的系数,它可以是0或者1。这样式(1.2.3)也可以作为二进制数转换为十进制数的转换公式。

例 1.2.1 试将二进制数 $(1010110)_B$ 转换为十进制数。

解: 将每一位二进制数与其位权相乘,然后相加便得相应的十进制数。

$$(1010110)_B = 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = (86)_D$$

2. 二进制的优点

与十进制相比较,二进制具有一定的优点,因此它在计算机技术中被广泛采用。

(1) 二进制的数字电路简单可靠,所用元件少

二进制数的每一位只有0和1两个状态,可用任何具有两个不同稳定状态的元件来表示,如BJT的饱和与截止,继电器触点的闭合与断开,灯的亮和不亮等。只要规定其中一种状态表示1,另一种状态表示0,就可以表示二进制数。这样,数码的存储、分析和传输,就可以用简单而可靠的方式进行。

(2) 二进制的基本运算规则简单,运算操作方便

二进制只有两个数码0和1,基本运算规则简单。

① 下标B(Binary)表示二进制。

但是,采用二进制也有缺点。用二进制表示一个数时,位数太多,例如,十进制数 49 表示为二进制时,即为 **110001**,不便于书写和阅读,使用不方便。因此,通常在阅读和书写时采用十进制数。在送入计算机时,计算机先将十进制数转换成数字系统能接受的二进制数;而在运算结束后,再将二进制转换为十进制数,输出十进制结果。

3. 二进制的波形表示

在数字电路中,二值数据常用数字波形来表示。图 1.2.1 所示为 16 位数据 **0010111100111010** 的波形表示。数据的波形表示比较直观,也便于用示波器进行监视。图 1.2.2 所示为一计数器的波形,图中最左列标出了二进制数的位权(2^0 、 2^1 、 2^2 、 2^3)以及最低位(Least Significant Bit, LSB)和最高位(Most Significant Bit, MSB),最后一行标出了从 0 到 15 的等效十进制数。



图 1.2.1 16 位数据的波形表示

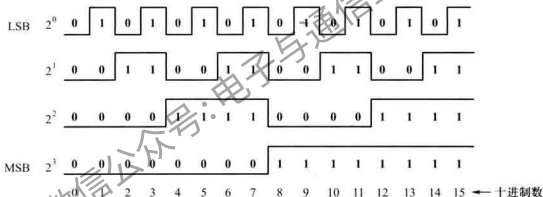


图 1.2.2 用二进制数表示 0~15 波形图

从图 1.2.2 还可看出,每一位的波形均为方波,其占空比均为 50%,但从低位到高位,每一波形的频率逐位减半。

4. 二进制数据的传输

二进制数据从一处传输到另一处,可以采用串行的方式或并行的方式。对于串行的方式,一组数据在时钟脉冲的控制下逐位传送。串行方式所需的设备简单,只需一根导线和一共同接地端即可。两台计算机之间,或计算机通过电话线与网络连接均采用这种方式。

二进制数据作串行传输的示意图如图 1.2.3 所示,图 1.2.3(a)表示二进制数据 **00110110** 从计算机 A 串行传送到计算机 B。图 1.2.3(b)是在时钟脉冲 CP 的控制下,数据由最高位 MSB 到最低位 LSB 依次传输的波形图。注意每传送一位数需要一个时钟周期,并且在时钟脉冲的下降沿完成。

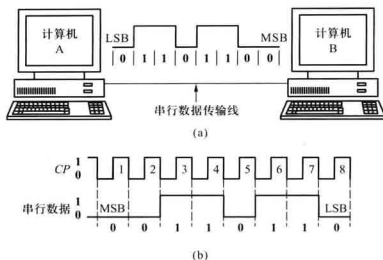


图 1.2.3 二进制数据的串行传输

(a) 两台计算机之间的串行通信 (b) 二进制数据的串行表示

若要求传输速度快,则可采用并行传输的方式,即将一组二进制数据同时传送。图 1.2.4 为并行传输数据的示意图。图 1.2.4(a)所示为一台打印机从一台计算机以 8 位数据并行的方式获取数据。传输 8 位数据所需的时间为一个时钟脉冲的周期,只有串行传输时间的 $\frac{1}{8}$ 。但所需设备复杂,需用 8 条传输线和其他部件。并行传输在数字系统中是一种常用的技术。

1.2.3 十-二进制之间的转换

既然同一个数可以用二进制和十进制两种不同形式来表示,那么两者之间必然有一定的转换关系。前面已经介绍了二进制数转换为十进制数的方法,是将每位二进制数与其权相乘,然后相加便得到相应的十进制数。十进制数转换为二进制数时,整数部分和小数部分的方法不同,下面分别介绍。

对于整数部分可写成

$$(N)_D = b_n \times 2^n + b_{n-1} \times 2^{n-1} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 \quad (1.2.4)$$

式中 $b_n, b_{n-1}, \cdots, b_1, b_0$ 是二进制数各位数字。将等式两边分别除以 2,得

$$\frac{1}{2}(N)_D = b_n \times 2^{n-1} + b_{n-1} \times 2^{n-2} + \cdots + b_1 \times 2^0 + \frac{b_0}{2} \quad (1.2.5)$$

由此可知,将十进制数除以 2,其余数为 b_0 。得到的商为

$$b_n \times 2^{n-1} + b_{n-1} \times 2^{n-2} + \cdots + b_1$$

可以写成

$$b_n \times 2^{n-1} + b_{n-1} \times 2^{n-2} + \cdots + b_1 = 2 \left(b_n \times 2^{n-2} + b_{n-1} \times 2^{n-3} + \cdots + b_2 + \frac{b_1}{2} \right) \quad (1.2.6)$$

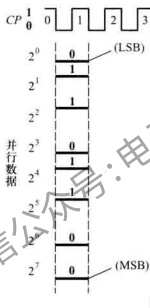
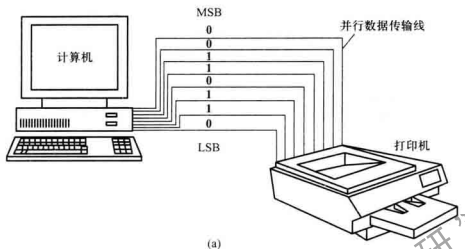


图 1.2.4 并行传输数据的示意图

(a) 计算机与打印机之间的并行通信 (b) 二进制数据的并行表示

将式(1.2.6)再除以2,其余数为 b_1 。不难推知,将十进制整数每除以一次2,就可根据余数得到二进制的1位数字。因此,只要连续除以2直到商为0,就可由所有的余数求出二进制数。

例 1.2.2 将十进制数 $(37)_{10}$ 转换为二进制数。

解: 根据上述原理,可将 $(37)_{10}$ 按如下的步骤转换为二进制数:

$$\begin{array}{rcl}
2 & \overline{) 37} & \cdots \cdots \cdots \text{余 } 1 \cdots \cdots b_0 \\
2 & \overline{) 18} & \cdots \cdots \cdots \text{余 } 0 \cdots \cdots b_1 \\
2 & \overline{) 9} & \cdots \cdots \cdots \text{余 } 1 \cdots \cdots b_2 \\
2 & \overline{) 4} & \cdots \cdots \cdots \text{余 } 0 \cdots \cdots b_3 \\
2 & \overline{) 2} & \cdots \cdots \cdots \text{余 } 0 \cdots \cdots b_4 \\
2 & \overline{) 1} & \cdots \cdots \cdots \text{余 } 1 \cdots \cdots b_5 \\
& 0 &
\end{array}$$

由上得

$$(37)_D = (100101)_B$$

当十进制数较大时,不必逐次除 2,而是将十进制数和与其相当的 2 的幂项对比,使转换过程得到简化。

例 1.2.3 将 $(133)_D$ 转换为二进制数。

解: 由于 2^7 为 128,而 $133-128=5=2^2+2^0$,所以对应二进制数 $b_7=1, b_2=1, b_0=1$,其余各系数均为 0,所以得

$$(133)_D = (10000101)_B$$

值得指出,多数计算机或数字系统中只处理 4、8、16、32 或 64 位等的二进制数据,因此,数据的位数需配成规格化的位数,如例 1.2.2 中转换结果为 100101,如将它配成 8 位,则相应的高幂项应填以 0,其值不变,即

$$100101 = 00100101$$

对于二进制的小数部分可写成

$$(N)_B = b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + \cdots + b_{-(n-1)} \times 2^{-(n-1)} + b_{-n} \times 2^{-n} \quad (1.2.7)$$

将上式两边分别乘以 2,得

$$2 \times (N)_B = b_{-1} \times 2^0 + b_{-2} \times 2^{-1} + \cdots + b_{-(n-1)} \times 2^{-(n-2)} + b_{-n} \times 2^{-(n-1)} \quad (1.2.8)$$

由此可见,将十进制小数乘以 2,所得乘积的整数即为 b_{-1} 。不难推知,将十进制小数每次去掉上次所得积中的整数再乘以 2,直到满足误差要求进行“四舍五入”为止,就可完成将十进制小数转换成二进制小数。

例 1.2.4 将 $(0.706)_D$ 转换为二进制数,要求其误差不大于 2^{-10} 。

解: 按上面介绍的方法计算,可得 $b_{-1}, b_{-2}, \cdots, b_{-9}$ 如下:

$$0.706 \times 2 = 1.412 \cdots \cdots 1 \cdots \cdots b_{-1}$$

$$0.412 \times 2 = 0.824 \cdots \cdots 0 \cdots \cdots b_{-2}$$

$$0.824 \times 2 = 1.648 \cdots \cdots 1 \cdots \cdots b_{-3}$$

$$0.648 \times 2 = 1.296 \cdots \cdots 1 \cdots \cdots b_{-4}$$

$$0.296 \times 2 = 0.592 \cdots \cdots 0 \cdots \cdots b_{-5}$$

$$0.592 \times 2 = 1.184 \cdots \cdots 1 \cdots \cdots b_{-6}$$

$$0.184 \times 2 = 0.368 \cdots \cdots 0 \cdots \cdots b_{-7}$$

$$0.368 \times 2 = 0.736 \cdots \cdots \mathbf{0} \cdots \cdots b_{-8}$$

$$0.736 \times 2 = 1.472 \cdots \cdots \mathbf{1} \cdots \cdots b_{-9}$$

$$0.472 \times 2 = 0.944 \cdots \cdots \mathbf{0} \cdots \cdots b_{-10}$$

由于最后的小数大于 0.5, 根据“四舍五入”的原则, b_{-10} 应为 **1**。所以, $(0.706)_D = (0.1011010011)_H$, 其误差 $\varepsilon < 2^{-10}$ 。

1.2.4 十六进制和八进制

对于同一个数, 用二进制数表示比用十进制数表示需要的位数多, 不便书写和记忆, 因此在计算机资料中常采用十六进制数或八进制数来表示。

1. 十六进制

十六进制数采用 16 个数码, 分别为 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F。其中 A、B、C、D、E、F 依次相当于十进制数中的 10、11、12、13、14、15。十六进制数是“逢十六进一”, 见表 1.2.1, 它是以 16 为基数的计数体制。

根据式(1.2.2), 任意十六进制数可表达如下:

$$(N)_H^{(1)} = \sum_{i=0}^n K_i \times 16^i \quad (1.2.9)$$

式中 K_i 为基数 16 的第 i 次幂的系数。

例 1.2.5 将十六进制数 4E6 转换为十进制数。

解: $(4E6)_H = 4 \times 16^2 + 14 \times 16^1 + 6 \times 16^0 = 1254$

2. 十六-二进制之间的转换

4 位二进制数有 16 个状态, 而 1 位十六进制数有 16 个不同的数码, 因此二进制转换为十六进制非常简单, 以小数点为基准, 整数部分从右到左每 4 位一组, 不足 4 位的在高位补 0; 小数部分从左到右每 4 位一组, 不足 4 位的在低位补 0。每 4 位一组的二进制数就表示 1 位十六进制数。

例 1.2.6 将二进制数 10111001011.01001 转换为十六进制数。

解: 将 4 位二进制数分为一组, 用相应的十六进制数代替即得

$$(0101 \ 1100 \ 1011 \ 0100 \ 1000)_B = (5CB.48)_H$$

例 1.2.7 将十六进制数 F15.6 转换为二进制数。

解: 将每位十六进制数用 4 位二进制数代替即得相应的二进制数

$$(F15.6)_H = (1111 \ 0001 \ 0101 \ 011)_B$$

3. 八进制

同样, 八进制数由 0~7 共 8 个数码表示, 且“逢八进一”。任意八进制可表示为

① 下标 H(Hexadecimal) 表示十六进制。(59)_H 也可写成 59H。

$$(N)_8^{①} = \sum_{i=-\infty}^{\infty} K_i \times 8^i \quad (1.2.10)$$

式中 K_i 为基数 8 的第 i 次幂的系数。

4. 八-二进制之间的转换

同理,对于八进制数,可将 3 位二进制数分为一组,对应于 1 位八进制数。如将例 1.2.6 中的二进制数转换为八进制数,则可写成

$$(010 \ 111 \ 001 \ 011 \ 010 \ 010)_2 = (2713.22)_8$$

至于十进制数转换为十六进制数,可先将十进制数转换为二进制数,再由二进制数转换为十六进制数。

为便于对照,将十进制、二进制、八进制及十六进制之间的关系列于表 1.2.1 中。^①

表 1.2.1 几种数制之间的关系对照表

十进制数	二进制数	八进制数	十六进制数
0	0 0 0 0 0	0	0
1	0 0 0 0 1	1	1
2	0 0 0 1 0	2	2
3	0 0 0 1 1	3	3
4	0 0 1 0 0	4	4
5	0 0 1 0 1	5	5
6	0 0 1 1 0	6	6
7	0 0 1 1 1	7	7
8	0 1 0 0 0	10	8
9	0 1 0 0 1	11	9
10	0 1 0 1 0	12	A
11	0 1 0 1 1	13	B
12	0 1 1 0 0	14	C
13	0 1 1 0 1	15	D
14	0 1 1 1 0	16	E
15	0 1 1 1 1	17	F
16	1 0 0 0 0	20	10
17	1 0 0 0 1	21	11
18	1 0 0 1 0	22	12
19	1 0 0 1 1	23	13
20	1 0 1 0 0	24	14

① 下标 O(Octal) 表示八进制。为了更清楚起见,某些文献中也用下标 2,8,10 及 16 分别表示二进制、八进制、十进制及十六进制。

复习思考题

- 1.2.1 为什么在计算机或数字系统中通常采用二进制数？
- 1.2.2 在二进制数中，其位权的规律是什么？
- 1.2.3 十六进制数主要用于何种场合？
- 1.2.4 二进制与十六进制之间如何进行转换？

1.3 二进制数的算术运算

在数字电路中，0 和 1 既可以表示逻辑状态，又可以表示数量大小。当表示数量时，两个二进制数可以进行算术运算。下面介绍无符号二进制数和有符号二进制数的算术运算。

1.3.1 无符号二进制数的算术运算

二进制数的加、减、乘、除四种运算的运算规则与十进制数类似，两者唯一的区别在于进位或借位规则不同。

1. 二进制加法

无符号二进制的加法规则是：

$$0+0=0, 0+1=1, 1+1=\boxed{1}0$$

方框中的 1 是进位位，表示两个 1 相加“逢二进一”。

例 1.3.1 计算两个二进制数 1010 和 0101 的和。

解：

$$\begin{array}{r} 1010 \\ + 0101 \\ \hline 1111 \end{array}$$

所以

$$1010+0101=1111$$

无符号二进制数的加法运算是基础，数字系统中的各种算术运算都将通过它来进行。

2. 二进制减法

无符号二进制数的减法规则是：

$$0-0=0, 1-1=0, 1-0=1, 0-1=\boxed{1}1$$

方框中的 1 是借位位，表示 0 减 1 时不够减，向高位借 1。

例 1.3.2 计算两个二进制数 1010 和 0101 的差。

解：

$$\begin{array}{r}
 1010 \\
 -0101 \\
 \hline
 0101
 \end{array}$$

所以

$$1010 - 0101 = 0101$$

由于无符号二进制数中无法表示负数,则要求被减数一定大于减数。

3. 乘法运算和除法运算

无符号二进制数的乘法规则是:

$$0 \times 0 = 0, 0 \times 1 = 0, 1 \times 0 = 0, 1 \times 1 = 1$$

无符号二进制数的除法规则是:

$$0 \div 1 = 0, 1 \div 1 = 1$$

注意,除数不能为0,否则无意义。

例 1.3.3 计算两个二进制数 1010 和 0101 的积。

解:

$$\begin{array}{r}
 1010 \\
 \times 0101 \\
 \hline
 1010 \\
 0000 \\
 1010 \\
 0000 \\
 \hline
 110010
 \end{array}$$

所以

$$1010 \times 0101 = 110010$$

由上述运算过程可见,乘法运算是由左移被乘数与加法运算组成的。

例 1.3.4 计算两个二进制数 1010 和 111 之商。

解:

$$\begin{array}{r}
 1011 \\
 111 \overline{) 1010} \\
 \underline{111} \\
 1100 \\
 \underline{111} \\
 1010 \\
 \underline{111} \\
 11 \cdots \cdots \text{余数}
 \end{array}$$

所以

$$1010 \div 111 = 1.011 \quad \text{余 } 11$$

由上述运算过程可见,除法运算是由右移除数与减法运算组成的。

1.3.2 带符号二进制数的减法运算

前面只考虑了二进制数的正数,当涉及负数时,就要用有符号的二进制数表示。在定点运算的情况下,二进制数的最高位(即最左边的位)表示符号位,且用 0 表示正数,用 1 表示负数。其余部分为数值位。例如

$$(+11)_D = (0 \ 1011)_B$$

$$(-11)_D = (1 \ 1011)_B$$

以上是用原码的形式表示数值位。在数字电路或系统中,为简化电路,常将负数用补码表示,以便将减法运算变为加法运算。下面首先介绍补码的概念,然后举例说明负数的求补方法及减法运算,同时为了便于得到补码,引入反码的概念。

1. 二进制数的补码表示

若基数为 R ,位数为 n 的原码 N ,其补码为

$$N_{\text{补}} = R^n - N \quad (1.3.1)$$

首先以常用的十进制数为例,2 和 46 的补码分别为 $10-2=8$ 和 $10^2-46=54$ 。

例 1.3.5 利用补码分别计算出 $8-2$ 和 $82-46$ 。

解: 根据式(1.3.1)可知 $-N = N_{\text{补}} - R$,所以

$$8-2 = 8+2_{\text{补}}-10 = 8+8-10 = 6$$

$$82-46 = 82+46_{\text{补}}-10^2 = 82+54-100 = 36$$

上面计算的是无符号十进制数的补码。对于无符号二进制数,同样可以利用式(1.3.1)进行补码的计算。当考虑负数情况时,带符号二进制数补码的计算方法如下:

补码或反码的最高位为符号位,正数为 0,负数为 1。

当二进制数为正数时,其补码、反码与原码相同。

当二进制数为负数时,将原码的数值位逐位求反(即得到反码),然后在最低位加 1 得到补码。

例 1.3.6 分别计算出 $A=+6$ 和 $B=-6$ 的 4 位二进制的原码、反码和补码。

解: A 和 B 的绝对值均为 6。除最高位为符号位外,还有 3 位为数值位。其原码、反码和补码分别为

$$A_{\text{原}} = 0 \ 110$$

$$B_{\text{原}} = 1 \ 110$$

$$A_{\text{反}} = 0 \ 110$$

$$B_{\text{反}} = 1 \ 001$$

$$A_{\text{补}} = 0 \ 110$$

$$B_{\text{补}} = 1 \ 010$$

4 位带符号的二进制数的原码、反码和补码对照表如表 1.3.1 所示。对于 -0 和 $+0$,原码和反码用不同的 4 位二进制数表示,而补码则是相同的。因此,它们表示的数值范围分别为,原码

是 $-7 \sim +7$,反码也是 $-7 \sim +7$,补码是 $-8 \sim +7$ 。由此可以推知,对于 n 位带符号的二进制数的原码、反码和补码的数值范围分别为:

原码 $-(2^{n-1}-1) \sim +(2^{n-1}-1)$

反码 $-(2^{n-1}-1) \sim +(2^{n-1}-1)$

补码 $-2^{n-1} \sim +(2^{n-1}-1)$

表 1.3.1 4 位二进制数原码、反码、补码对照表

十进制数	二进制数		
	原码	反码	补码
-8	—	—	1 0 0 0 ¹
-7	1 1 1 1	1 0 0 0	1 0 0 1
-6	1 1 1 0	1 0 0 1	1 0 1 0
-5	1 1 0 1	1 0 1 0	1 0 1 1
-4	1 1 0 0	1 0 1 1	1 1 0 0
-3	1 0 1 1	1 1 0 0	1 1 0 1
-2	1 0 1 0	1 1 0 1	1 1 1 0
-1	1 0 0 1	1 1 1 0	1 1 1 1
-0	1 0 0 0	1 1 1 1	0 0 0 0
+0	0 0 0 0	0 0 0 0	0 0 0 0
+1	0 0 0 1	0 0 0 1	0 0 0 1
+2	0 0 1 0	0 0 1 0	0 0 1 0
+3	0 0 1 1	0 0 1 1	0 0 1 1
+4	0 1 0 0	0 1 0 0	0 1 0 0
+5	0 1 0 1	0 1 0 1	0 1 0 1
+6	0 1 1 0	0 1 1 0	0 1 1 0
+7	0 1 1 1	0 1 1 1	0 1 1 1

2. 二进制补码的减法运算

采用补码的形式,可以很方便地进行带符号二进制数的减法运算。减法运算的原理是减去一个正数相当于加上一个负数,即 $A-B=A+(-B)$,对 $(-B)$ 求补码,然后进行加法运算。至于乘法和除法运算,从 1.3.1 节介绍知道,可以采用移位与加法或减法的组合完成。

进行二进制补码的加法运算时,必须注意被加数补码与加数补码的位数相等,即让两个二进制数补码的符号位对齐。通常两个二进制数的补码采用相同的位数表示。

例 1.3.7 试用 4 位二进制补码计算 $5-2$ 。

解: 因为 $(5-2)_{\text{十}} = (5)_{\text{十}} + (-2)_{\text{十}}$

$$= 0101 + 1110$$

$$= 0011$$

所以

$$5-2=3$$

$$0101$$

$$+ 1110$$

$$[1]0011$$

自动丢弃

两个二进制补码相加时,方框中的1是进位位,在计算中会自动丢失,因为运算是以4位二进制补码表示的,计算结果仍然保留4位数。

3. 溢出

例 1.3.8 试用4位二进制补码计算 $5+7$ 。

解: 因为

$$(5+7)_{\text{十}} = (5)_{\text{十}} + (7)_{\text{十}}$$

$$= 0101 + 0111$$

$$= 1100$$

$$0101$$

$$+ 0111$$

$$[1]100$$

计算结果1100表示-4,而实际正确的结果应该为12。错误产生的原因在于4位二进制补码中,有3位是数值位,它所表示的范围为-8~+7,而本题的结果需要4位数数值位表示,因而产生溢出。解决溢出的办法是进行位扩展,即用5位以上的二进制补码表示,就不会产生溢出了。

4. 溢出的判别

两个符号相反的数相加不会产生溢出,但两个符号相同的数相加有可能产生溢出。如何判断是否产生溢出?根据下列同符号4位二进制补码的计算结果可以推知。

$$\begin{array}{r} +4 \quad 0100 \\ +) +3 \quad 0011 \\ \hline +7 \quad [0]0111 \end{array}$$

(a)

$$\begin{array}{r} -5 \quad 1011 \\ +) -3 \quad 1101 \\ \hline -8 \quad [1]1000 \end{array}$$

(b)

$$\begin{array}{r} +2 \quad 0010 \\ +) +6 \quad 0110 \\ \hline +8 \quad [0]1000 \end{array}$$

(c)

$$\begin{array}{r} -3 \quad 1101 \\ +) -6 \quad 1010 \\ \hline -9 \quad [1]0111 \end{array}$$

(d)

4位二进制补码表示的数值范围为-8~+7,所以(a)和(b)没有产生溢出,结果是正确的。(c)和(d)的运算结果应分别是+8和-9,均超过了允许的范围,产生溢出。比较4种情况可以看出,当方框中的进位位与和数的符号位(即 b_3 位)相反时,则运算结果是错误的,产生溢出。

复习思考题

- 1.3.1 为什么说二进制的加法运算是算术运算基础?
- 1.3.2 二进制数乘法、除法运算过程各有什么规律?
- 1.3.3 说明反码与补码之间的关系。
- 1.3.4 简要说明由加补码完成减法运算的原理。

1.3.5 说明溢出产生的原因及解决的办法。

1.4 二进制代码

数值和文字符号(包括控制符)是数字系统中处理最多的两类信息。数值信息的表示方法如前所述。文字符号信息通常也采用二进制数码表示,这些数码并不表示数量的大小,仅仅区别不同事物而已。这些特定的二进制数码称为代码。以一定的规则编制代码,用以表示十进制数值、字母、符号等的过程称为**编码**。将代码还原成所表示的十进制数、字母、符号等的过程称为**解码**或**译码**。

若所需编码的信息有 N 项,则需要的二进制数码的位数 n 应满足如下关系

$$2^n \geq N$$

1.4.1 二-十进制码

二-十进制码就是用 4 位二进制数来表示 1 位十进制数中的 0~9 十个数码,即二进制编码的十进制码(Binary-Coded-Decimal,BCD 码)。4 位二进制数有 16 种不同的组合方式,即 16 种代码,根据不同的规则从中选择 10 种来表示十进制的 10 个数码。其方案有很多种。表 1.4.1 列出了几种常用的 BCD 码。

表 1.4.1 几种常见的 BCD 码

十进制数	有权码			无权码	
	8421 码	2421 码	5421 码	余 3 码	余 3 循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

8421BCD 码是最常用的一种 BCD 码。它是由 4 位自然二进制数 0000(0)到 1111(15)16 种组合的前 10 种组合,即 0000(0)~1001(9)构成,其余 6 种组合是无效的。其编码中每位的值都是固定数,称为**位权**。 b_3 位的权为 $2^3=8$, b_2 位的权为 $2^2=4$, b_1 位的权为 $2^1=2$, b_0 位的权为 $2^0=1$,因此称为 8421BCD 码。它属于有权码。

2421 码也是有权码。对应 b_3 、 b_2 、 b_1 和 b_0 的权分别是 2、4、2 和 1。它的特点是,将任意一个十进制数 N 的代码各位取反,所得代码正好表示 N 的 9 的补码。例如 2 的代码 0010 各位取反所得代码 1101 正好是 $9-2=7$ 的代码。这种特性称为自补性。具有自补特性的代码称为自补码。

5421 码也是有权码,它各位的权依次为 5、4、2、1。在一般情况下,有权码的十进制数与二进制数之间可用下式来表示

$$(N)_B = W_3 b_3 + W_2 b_2 + W_1 b_1 + W_0 b_0 \quad (1.4.1)$$

式中 $W_3 \sim W_0$ 为二进制码中各位的权。

余 3 码是自补码,与 2421 码有类似的自补性。当两个十进制数之和是 10 时,相应的二进制数之和是 16。例如 1 和 9、2 和 8、 $\dots$ 、6 和 4。该特点便于求 10 的补码。余 3 码也是无权码,它的每一位没有一定的权值,不能用式 (1.4.1) 来表示其编码关系,但其编码可以由 8421 码加 3(0011) 得出。

余 3 循环码也是一种无权码,它的特点是具有相邻性,任意两个相邻代码之间仅有 1 位取值不同,例如 4 和 5 两个代码 0100 和 1100 仅 b_3 不同。余 3 循环码可以看成是将格雷码首尾各 3 种状态去掉后得到的。下面介绍格雷码。

1.4.2 格雷码

格雷码 (Gray Code) 也是一种常见的无权码,表 1.4.2 给出了 4 位格雷码的编码顺序。它也具有相邻性,即两个相邻代码之间仅有 1 位取值不同,并且 0 和最大数 (2^n-1) 之间也只有 1 位不同,因此它是一种循环码。格雷码的这个特点使它在代码形成和传输时引起的误差较小。因而常用于将模拟量转换成用连续二进制数序列表示数字量的系统中。当模拟量发生微小变化而引起数字量从一位变化到相邻位时,例如从十进制数的 3 到 4,格雷码变化是从 0010 到 0110,只有 b_2 位从 0 变成 1,其余 3 位保持不变。如果对于自然二进制码,其变化是从 0011 到 0100,有 3 位发生变化,如果 b_2 位从 0 到 1 变化所需的时间,比 b_1 和 b_0 从 1 变到 0 的时间长,则在转换过程中,会出现瞬间错误码 0000。而格雷码可以避免错误码数的出现。

格雷码的缺点是不能直接进行算术运算。这是因为格雷码是无权码,其每一位的权值不是固定的。

表 1.4.2 格 雷 码

十进制数	二进制码				格雷码			
	b_3	b_2	b_1	b_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0

续表

十进制数	二进制码				格雷码			
	b_3	b_2	b_1	b_0	G_3	G_2	G_1	G_0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

格雷码和二进制码之间经常需要转换,具体转换方法如下。

1. 二进制码到格雷码的转换

- (1) 格雷码的最高位(最左边)与二进制码的最高位相同。
- (2) 从左到右,逐一将二进制码相邻的2位相加(舍去进位),作为格雷码的下一位。

例 1.4.1 将二进制码 1011 转换成格雷码。

解: 转换过程如下:

$$\begin{array}{ccccccc}
 1 & \rightarrow & 0 & \rightarrow & 1 & \rightarrow & 1 \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow \\
 1 & & 1 & & 1 & & 0
 \end{array}
 \begin{array}{l}
 \text{二进制码} \\
 \\
 \text{格雷码}
 \end{array}$$

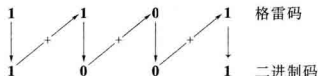
所以,格雷码为 1110。

2. 格雷码到二进制码的转换

- (1) 二进制码的最高位(最左边)与格雷码的最高位相同。
- (2) 将产生的每一位二进制码,与下一位相邻的格雷码相加(舍去进位),作为二进制码的下一位。

例 1.4.2 将格雷码 1101 转换成二进制码。

解: 转换过程如下:



所以,二进制码为 **1001**。

格雷码和二进制码相互转换的另一种方法是用**异或**表达式(参见 4.2.1 节)。

1.4.3 ASCII 码

计算机不仅用于处理数字,而且用于处理字母、符号等文字信息。人们通过键盘上的字母、符号和数值向计算机发送数据和指令,每一个键符可用一个二进制码来表示,美国标准信息交换码(American Standard Code for Information Interchange, ASCII)是目前国际上最通用的一种字符码。它是用 7 位二进制码来表示 128 个十进制数、英文大小写字母、控制符、运算符以及特殊符号,如表 1.4.3A 所示,其中一些字符的含义如表 1.4.3B 所示。

表 1.4.3A ASCII 码

$b_5b_4b_3b_2b_1b_0$	$A_6A_5A_4A_3A_2A_1A_0$							
	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
0 0 0 0	NUL	DLE	SP	0	@	P	^	p
0 0 0 1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0	STX	DC2	"	2	B	R	b	r
0 0 1 1	ETX	DC3	#	3	C	S	c	s
0 1 0 0	EOF	DC4	\$	4	D	T	d	t
0 1 0 1	ENO	NAK	%	5	E	U	e	u
0 1 1 0	ACK	SYN	&	6	F	V	f	v
0 1 1 1	BEL	ETB	'	7	G	W	g	w
1 0 0 0	BS	CAN	(	8	H	X	h	x
1 0 0 1	HT	EM	)	9	I	Y	i	y
1 0 1 0	LF	SUB	*	:	J	Z	j	z
1 0 1 1	VT	ESC	+	;	K	[	k	
1 1 0 0	FF	FS	,	<	L	\	l	
1 1 0 1	CR	GS	-	=	M	]	m	
1 1 1 0	SO	RS	.	>	N	^	n	~
1 1 1 1	SI	US	/	?	O	_	o	DEL

表 1.4.3B ASCII 码中各字符的含义

字符	含 义	字符	含 义
NUL	Null 空白	DC1	Device control 1 设备控制 1
SOH	Start of heading 标题开始	DC2	Device control 2 设备控制 2
STX	Start of text 文本开始	DC3	Device control 3 设备控制 3
ETX	End of text 文本结束	DC4	Device control 4 设备控制 4
EOT	End of transmission 传输结束	NAK	Negative acknowledge 否认
ENQ	Enquiry 询问	SYN	Synchronous idle 同步空转
ACK	Acknowledge 确认	ETB	End of transmission block 块传输结束
BEL	Bell 报警	CAN	Cancel 取消
BS	Backspace 退一格	EM	End of medium 纸尽
HT	Horizontal tab 水平列表	SUB	Substitute 替换
LF	Line feed 换行	ESC	Escape 脱离
VT	Vertical tab 垂直列表	FS	File separator 文件分隔符
FF	Form feed 走纸	GS	Group separator 组分隔符
CR	Carriage return 回车	RS	Record separator 记录分隔符
SO	Shift out 移出	US	Unit separator 单元分隔符
SI	Shift in 移入	SP	Space 空格
DLE	Data link escape 数据链路换码	DEL	Delete 删除

复习思考题

- 1.4.1 什么叫 BCD 码？试列举几种常用的 BCD 码。
- 1.4.2 为什么 8421BCD 码用的较普遍？
- 1.4.3 格雷码有什么特点，用于什么场合？
- 1.4.4 格雷码与二进制码之间如何进行转换？
- 1.4.5 什么是 ASCII 码？试说明它的用途。

1.5 二值逻辑变量与基本逻辑运算

当 0 和 1 表示逻辑状态时，两个二进制数码按照某种指定的因果关系进行的运算称为逻辑运算。逻辑运算与算术运算完全不同，它所使用的数学工具是逻辑代数（又称为布尔代数）。与普通代数一样，它由逻辑变量和逻辑运算组成。变量可以用 A, B, C, x, y, z 等字母组成。所不同的是，在普通代数中，变量的取值可以是任意的，而在逻辑代数中，逻辑变量只有两个可取的值，即 0 和 1，因而称为二值逻辑变量。这里，0 和 1 并不表示数量的大小，而是用来表示完全对立的

逻辑状态。

在逻辑代数中,有与、或、非三种基本的逻辑运算。众所周知,运算是一种函数关系,它可以用语言描述,亦可用逻辑代数表达式描述,还可用表格或图形来描述。输入逻辑变量所有取值的组合与其对应的输出逻辑函数值构成的表格,称为真值表。用规定的逻辑符号表示的图形称为逻辑图。下面分别讨论三种基本的逻辑运算。

1. 与运算

图 1.5.1(a)所示为一个简单的与逻辑电路,电源 E 通过开关 A 和 B 向灯供电,只有 A 与 B 同时闭合时,灯才亮。 A 和 B 中只要有一个断开或者两个均断开时,则灯不亮。在这个电路中,开关 A 、 B 与灯 L 的逻辑关系是:“只有当一件事的几个条件全部具备之后,这件事才发生”。这种关系称为与逻辑。如果用二值逻辑 0 和 1 来表示开关和灯的状态,设开关断开和灯不亮均用 0 表示,而开关闭合和灯亮均用 1 表示,则可得出其真值表,如表 1.5.1 所示。其中 L 表示灯的状态。若用逻辑表达式来描述,则可写为

$$L = A \cdot B \quad (1.5.1)$$

式中小圆点“ $\cdot$ ”表示 A 、 B 的与运算,也称为逻辑乘。在不致引起混淆的前提下,乘号“ $\cdot$ ”被省略。在某些文献中,也采用符号 $\wedge$ 、 $\cap$ 表示与运算。能实现与运算的逻辑电路称为与门,其逻辑符号^①如图 1.5.1(b)和图 1.5.1(c)所示。图 1.5.1(b)所示为特异形符号,图 1.5.1(c)所示为矩形符号。

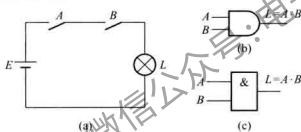


图 1.5.1 与逻辑运算

(a) 电路图 (b) 特异形符号 (c) 矩形符号

表 1.5.1 与逻辑真值表

A	B	$L = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

2. 或运算

图 1.5.2(a)所示为一个简单的或逻辑电路,电源 E 通过开关 A 或 B 向灯供电。只要开关 A 或 B 闭合或两者均闭合,则灯亮。而当 A 和 B 均断开时,则灯不亮。此电路中,开关 A 、 B 与灯 L 的逻辑关系是:“当一件事情的几个条件中只要有一个条件得到满足,这件事就会发生”。这种关系称为或逻辑。或是指 A 闭合或 B 闭合,即任一个条件具备的意思。仿照前述,可以得出用 0 、 1 表示的或逻辑真值表如表 1.5.2 所示。若用逻辑表达式来描述,则可写为

$$L = A + B \quad (1.5.2)$$

^① 电气和电子工程师协会(Institute of Electrical and Electronics Engineers,简称 IEEE)标准中,门电路的图形符号有两种,矩形符号和特异形符号(见附录 C)。中国国家标准(GB/T4728.12—1996)采用矩形符号。本书所用的逻辑图形符号为特异形符号。

式中符号“+”表示 A 、 B 的或运算,也表示逻辑加。在某些文献中,也采用符号 V 、 U 来表示或运算,能实现或运算的逻辑电路称为或门,其逻辑符号如图1.5.2(b)和图1.5.2(c)所示。图1.5.2(b)所示为特异形符号。图1.5.2(c)所示为矩形符号。

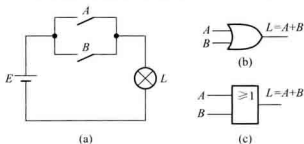


图 1.5.2 或逻辑运算

(a) 电路图 (b) 特异形符号 (c) 矩形符号

表 1.5.2 或逻辑真值表

A	B	$L = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

3. 非运算

如图1.5.3(a)所示,电压 V 通过一继电器触点向灯供电,NC为继电器 A 的动断触点,即通常 A 不通电,动断触点闭合,灯亮;而当 A 通电时,动断触点断开,灯不亮。由此可得出其逻辑关系为:“一件事情的发生是以其相反的条件为依据”。这种逻辑关系称为非逻辑。若用0和1来表示继电器和灯的状态, A 不通电和灯不亮用0表示,而 A 通电和灯亮用1表示,则得出非逻辑的真值表如表1.5.3所示。读者很容易看到 L 与 A 总是处于相反的逻辑状态。若用逻辑表达式来描述,则可写为

$$L = \bar{A} \quad (1.5.3)$$

式中字母 A 上方的短线“-”表示非运算。在某些文献中,也采用“~”、“ $\neg$ ”或“ $\neg$ ”来表示非运算。在逻辑运算中,通常将 A 称为原变量,而将 $\bar{A}$ 称为反变量或非变量。

能实现非运算的电路称为非门,也可以称为反相器,其逻辑符号如图1.5.3(b)、(c)所示,小圆圈表示非运算。

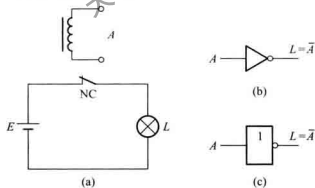


图 1.5.3 非逻辑运算

(a) 电路图 (b) 特异形符号 (c) 矩形符号

表 1.5.3 非逻辑真值表

A	$L = \bar{A}$
0	1
1	0

上述与、或逻辑运算可以推广到多变量的情况

$$L = A \cdot B \cdot C \cdots \quad (1.5.4)$$

$$L = A + B + C + \cdots \quad (1.5.5)$$

4. 几种常用逻辑运算

在实际逻辑运算中,除了与、或、非三种基本运算外,还经常使用一些其他的逻辑运算,例如与非、或非、异或和同或。

与非运算是与运算和非运算的组合。逻辑符号和真值表分别如图 1.5.4 和表 1.5.4 所示。逻辑表达式可写成

$$L = \overline{A \cdot B} \quad (1.5.6)$$

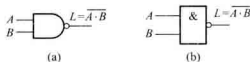


图 1.5.4 与非逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.4 与非逻辑真值表

A	B	L
0	0	1
0	1	1
1	0	1
1	1	0

或非运算是或运算和非运算的组合。逻辑符号和真值表分别如图 1.5.5 和表 1.5.5 所示。逻辑表达式可写成

$$L = \overline{A + B} \quad (1.5.7)$$

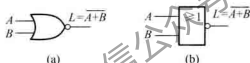


图 1.5.5 或非逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.5 或非逻辑真值表

A	B	L
0	0	1
0	1	0
1	0	0
1	1	0

异或的逻辑关系是:当两个输入状态相同时,输出为 0;当两个输入状态不同时,输出为 1。逻辑符号和真值表分别如图 1.5.6 和表 1.5.6 所示。逻辑表达式为

$$L = \overline{A}B + A\overline{B} = A \oplus B \quad (1.5.8)$$

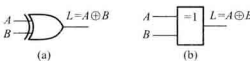


图 1.5.6 异或逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.6 异或逻辑真值表

A	B	L
0	0	0
0	1	1
1	0	1
1	1	0

同或和异或的逻辑关系刚好相反:当两个输入状态相同时,输出为 1;当两个输入状态不同

时,输出为**0**。逻辑符号和真值表分别如图 1.5.7 和表 1.5.7 所示。逻辑表达式为

$$L = AB + \bar{A}\bar{B} = A \odot B \quad (1.5.9)$$

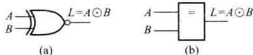


图 1.5.7 同或逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.7 同或逻辑真值表

A	B	L
0	0	1
0	1	0
1	0	0
1	1	1

复习思考题

- 1.5.1 试列举几种相关的实例说明与、或、非逻辑关系,并列写出 3 种逻辑运算的表达式。
- 1.5.2 根据真值表判断异或和同或具有怎样的逻辑关系?

1.6 逻辑函数及其表示方法

从上一节介绍的逻辑运算中可以知道,逻辑变量分为两种:输入逻辑变量和输出逻辑变量。描述输入逻辑变量和输出逻辑变量之间的因果关系称为逻辑函数。由于逻辑变量是只取**0**或**1**的二值逻辑变量,因此逻辑函数也是二值逻辑函数。

1.6.1 逻辑函数的几种表示方法

一般来说,一个比较复杂的逻辑电路,往往是受多种因素控制的,就是说有多个逻辑变量。输出变量与输入变量之间的逻辑函数的描述方法有真值表、逻辑函数表达式、逻辑图、波形图和卡诺图等。下面举一个简单实例介绍前四种逻辑函数的表示。

图 1.6.1 是一个控制楼梯照明灯的电路,单刀双掷开关 A 装在楼下, B 装在楼上,这样在楼下开灯后,可在楼上关灯;同样,也可在楼上开灯,而在楼下关灯。因为只有当两个开关都向上扳或向下扳时,灯才亮;而一个向上扳、另一个向下扳时,灯就不亮。

1. 真值表

将输入变量所有可能的取值与相应的函数值列成表格,就得到真值表。

图 1.6.1 电路的逻辑关系可用真值表来描述。设 L 表示灯的状态,即 $L=1$ 表示灯亮, $L=0$ 表示灯不亮。用 A 和 B 表示开关 A 和开关 B 的位置,用 **1** 表示开关向上扳,用 **0** 表示开关向下扳。 A 、 B 的取值有 4 种组合,00、01、10、11,对每一种组合确定输出 L 的值,则 L 与 A 、 B 逻辑关系的真值表如表 1.6.1 所示。

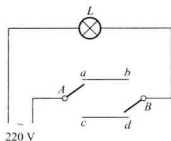


图 1.6.1 逻辑电路举例

表 1.6.1 图 1.6.1 的真值表

A	B	L
0	0	1
0	1	0
1	0	0
1	1	1

2. 逻辑函数表达式

逻辑表达式是用与、或、非等运算组合起来,表示逻辑函数与逻辑变量之间关系的逻辑代数式。

由表 1.6.1 所示真值表可知,在 A 、 B 状态的四种不同组合中,只有第一 ($A=B=0$) 和第四 ($A=B=1$) 两种组合才能使灯亮 ($L=1$)。这两种组合所对应的乘积项 $\bar{A}\bar{B}$ 和 AB 为 1,使 $L=1$ 。即输出 L 为两个乘积项之和,即

$$L = \bar{A}\bar{B} + AB \quad (1.6.1)$$

根据真值表写逻辑表达式的方法:逻辑变量之间是与的关系,而输出状态之间的组合则是或的关系。对于变量 A 、 B 或输出 L ,凡取 1 值的用原变量表示,取 0 值用反变量表示。

3. 逻辑图

用与、或、非等逻辑符号表示逻辑函数中各变量之间的逻辑关系所得到的图形称为逻辑图。

将式 (1.6.1) 中所有的与、或、非运算符用相应的逻辑符号代替,并按照逻辑运算的先后次序将这些逻辑符号连接起来,就得到图 1.6.1 所示电路所对应的逻辑图如图 1.6.2 (a) 所示。式 (1.6.1) 表示的是同或逻辑关系,为简便起见,也可以用同或逻辑符号表示,得到图 1.6.2 (b) 所示逻辑图。

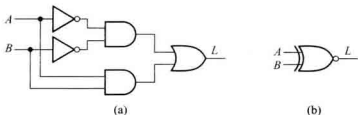


图 1.6.2 图 1.6.1 电路的逻辑图

(a) 由与、或、非逻辑符号构成的逻辑图 (b) 由同或逻辑符号构成的逻辑图

4. 波形图

对输入变量随时间变化的每一种取值,求出相应的输出值。并将输入和输出关系按时间顺

序依次排列得到的图形,称为波形图。

图 1.6.3 所示的波形图中,在 t_1 时间段内, A, B 输入端均为低电平 0, 根据式 (1.6.1) 或表 1.6.1 可知, 此时输出 L 为高电平 1。依照此方法, 可得出 t_2, t_3 和 t_4 时间段内输出 L 的波形图。从图中可以直观地看出, 对于同或逻辑关系, 只要输入 A 和 B 相同, 输出为 1; A 和 B 不相同, 输出为 0。

上述四种不同的表示方法所描述的是同一逻辑关系, 因此它们之间有着必然的联系, 可以从一种表示方法, 得到其他表示方法。

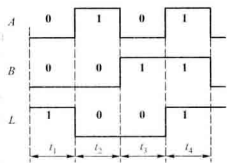


图 1.6.3 图 1.6.1 所示电路的波形图

1.6.2 逻辑函数表示方法之间的转换

逻辑函数的表示方法归纳起来有六种: 真值表、逻辑函数表达式、逻辑图、波形图、卡诺图和 HDL。卡诺图及 HDL 表示方法将在下一章介绍。

同一逻辑问题可以用六种不同方法描述。因此, 这些方法本质上是相通的, 可以相互转换。这里只介绍真值表与逻辑表达式及逻辑图之间的转换。

1. 真值表到逻辑图的转换

通常从给定的真值表不能直接得到逻辑图, 首先根据真值表写出逻辑表达式, 依照逻辑表达式画出逻辑图。

转换步骤:

- (1) 根据真值表写出逻辑表达式。
- (2) 用公式法或卡诺图法化简得到简化的逻辑表达式。
- (3) 根据逻辑表达式画出逻辑图。

例 1.6.1 已知真值表如表 1.6.2 所示, 试画出它的逻辑图。

解: (1) 写出真值表中使输出为 1 的所有输入变量乘积项, 凡取 1 值的用原变量表示, 取 0 值的用反变量表示。然后将这些乘积项相加, 即得到输出逻辑函数表达式。

当输入为 $A=0, B=1, C=1$ 时, 使乘积项 $\bar{A}BC=1$, 此时 $L=1$ 。同理乘积项 $AB\bar{C}$ 使 $L=1$ 。输出逻辑函数表达式就是这两个乘积项之和, 即

$$L = \bar{A}BC + AB\bar{C} \quad (1.6.2)$$

(2) 公式法或卡诺图法化简将在第 2 章介绍, 式 (1.6.2) 不需要化简。

(3) 将式 (1.6.2) 中所有的与、或、非运算符用相应的逻辑符号代替, 并按照逻辑运算的先后次序将它们连接起来, 得到如图 1.6.4 所示的逻辑图。

表 1.6.2 例 1.6.1 的真值表

A	B	C	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

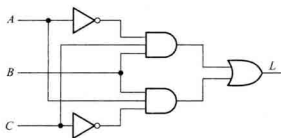


图 1.6.4 例 1.6.1 的逻辑图

2. 逻辑图到真值表的转换

从逻辑图不能直接得到真值表,转换步骤如下:

(1) 从逻辑图的输入端到输出端,逐级写出每个逻辑符号输出端的表达式,直到写出最后的输出变量的逻辑表达式。

(2) 化简变换,求简化的逻辑表达式。

(3) 将输入变量可能的取值逐个代入表达式进行计算,并将结果列表,即得真值表。

例 1.6.2 已知逻辑图如图 1.6.5 所示,列出它的真值表。

解: 从输入端开始写出每个逻辑符号输出端的表达式,得到最后的表达式为

$$L = \overline{A}B + A\overline{B}$$

该表达式不需要化简变换。

输入变量 A、B 取值有四种组合,其中 $\overline{A}B$ 和 $A\overline{B}$ 使 L 为 1,得到真值表如表 1.6.3 所示。

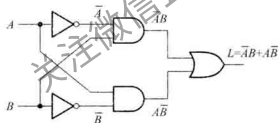


图 1.6.5 例 1.6.2 的逻辑图

表 1.6.3 例 1.6.2 的真值表

A	B	L
0	0	0
0	1	1
1	0	1
1	1	0

复习思考题

1.6.1 逻辑函数都有哪些表示方法?

1.6.2 你能根据给出的一种逻辑函数表示方法转换为其他的描述方式吗?

小 结

• 由于模拟信号具有连续性,实用上难于存储、分析和传输;数字电路或数字系统传输和处理 0 和 1 表示的信息,因此,较易克服这些困难。

• 用 0 和 1 可以组成二进制数表示数量的大小,也可以表示对立的两种逻辑状态。数字系统中常用二进制数来表示数值。所谓二进制是以 2 为基数的计数体制。

• 十六进制是二进制的简写,它是以 16 为基数的计数体制,常用于数字电子技术、微处理器、计算机和数据通信中。任意一种格式的数可以在十六进制、二进制和十进制之间相互转换。

• 与十进制数类似,二进制数也有加、减、乘、除四种运算,加法是各种运算的基础。二进制数可以用原码、反码或补码表示。在数字系统或计算机中常采用二进制补码表示有符号的数,并进行有关运算。

• 特殊二进制码常用来表示十进制数。如 8421 码、2421 码、5421 码、余 3 码、余 3 循环码、格雷码等。也有用 7 位二进制数来表示符号-数字混合码,如 ASCII 码。

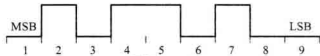
• 与、或、非是逻辑运算中的三种基本运算,其他的逻辑运算可以由这三种基本运算构成。数字逻辑是计算机的基础。逻辑函数的描述方法有真值表、逻辑函数表达式、逻辑图、波形图和卡诺图等。

习 题

1.1 数字信号与数字电路

1.1.1 试按表 1.1.1 所列的数字集成电路的分类为依据,指出下列 IC 器件属于何种集成度器件:(1) 微处理器;(2) 计数器;(3) 加法器;(4) 逻辑门;(5) 4 兆位存储器。

1.1.2 一数字信号波形如图题 1.1.2 所示,试问该波形所代表的二进制数是什么?



图题 1.1.2

1.1.3 试绘出下列二进制数的数字波形,设二进制数为串行方式,从左到右逻辑 1 的电压为 5 V,逻辑 0 的电压为 0 V。

(1) 001100110011 (2) 0111010 (3) 1111011101

1.1.4 一周期性数字波形如图题 1.1.4 所示,试计算:(1) 周期;(2) 频率;(3) 占空比。

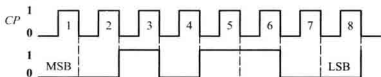


图题 1.1.4

1.2 数制

1.2.1 一数字波形如图题 1.2.1 所示,时钟频率为 4 kHz,试确定:

(1) 它所表示的二进制数;(2) 串行方式传送 8 位数据所需要的时间;(3) 以 8 位并行方式传送数据时需要的时间。



图题 1.2.1

1.2.2 将下列二进制数转换为十进制数:

- (1) $(1011)_2$ (2) $(10111)_2$ (3) $(101101)_2$ (4) $(1001011)_2$

1.2.3 将下列二进制数转换为十进制数:

- (1) $(0.101)_2$ (2) $(0.0011)_2$ (3) $(0.10101)_2$ (4) $(0.100111)_2$

1.2.4 将下列二进制数转换为十进制数:

- (1) $(10.01)_2$ (2) $(110.101)_2$ (3) $(1110.1001)_2$ (4) $(10111.01101)_2$

1.2.5 将下列二进制数转换为八进制和十六进制数:

- (1) $(101001)_2$ (2) $(1011.0101)_2$ (3) $(1010.11001)_2$ (4) $(110111.011101)_2$

1.2.6 将下列十进制数转换为二进制数、八进制数和十六进制数:

- (1) $(43)_{10}$ (2) $(59)_{10}$ (3) $(127)_{10}$ (4) $(234)_{10}$

1.2.7 将下列十进制数转换为二进制数、八进制数和十六进制数(要求二进制数保留小数点后 8 位):

- (1) $(0.9)_{10}$ (2) $(0.34)_{10}$ (3) $(0.028)_{10}$ (4) $(0.7182)_{10}$

1.2.8 将下列十进制数转换为二进制数、八进制数和十六进制数(要求转换误差不大于 2^{-4}):

- (1) $(4.8)_{10}$ (2) $(5.27)_{10}$ (3) $(254.35)_{10}$ (4) $(1002.456)_{10}$

1.2.9 将下列十进制数转换为二进制数:

- (1) $(77)_{10}$ (2) $(D4)_{16}$ (3) $(23F.45)_{16}$ (4) $(A040.51)_{16}$

1.2.10 将下列十六进制数转换为十进制数:

- (1) $(43)_{16}$ (2) $(103.2)_{16}$ (3) $(A3.0C)_{16}$ (4) $(A45D.0BC)_{16}$

1.2.11 试比较下列各数,并指出从大到小的排列顺序:

- (1) $(74)_{10}$ (2) $(1101101)_2$ (3) $(165)_{10}$ (4) $(10E)_{16}$

1.3 二进制数的算术运算

1.3.1 写出下列二进制数的原码、反码和补码:

- (1) $(+1110)_2$ (2) $(+10110)_2$ (3) $(-1110)_2$ (4) $(-10110)_2$

1.3.2 写出下列有符号二进制补码所表示的十进制数:

- (1) 01011 (2) 0010111 (3) 11101000 (4) 11011001

1.3.3 试用 8 位二进制补码表示下列十进制数:

- (1) +11 (2) +68 (3) -24 (4) -90

1.3.4 试用 8 位二进制补码计算下列各式,并用十进制数表示结果:

- (1) $12+9$ (2) $11-3$ (3) $-29-25$ (4) $-120+30$

1.4 二进制代码

1.4.1 将下列十进制数转换为 8421BCD 码:

- (1) 43 (2) 127 (3) 254.25 (4) 2.718

1.4.2 将下列数码分别作为自然二进制数和 8421BCD 码时, 求出相应的十进制数。

- (1) 10010111 (2) 100010010011 (3) 000101001001 (4) 10000100.10010001

1.4.3 将下列二进制数转换为格雷码:

- (1) 101 (2) 1011 (3) 11010 (4) 101101

1.4.4 将下列格雷码转换为二进制数:

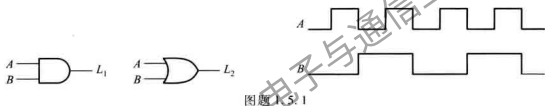
- (1) 110 (2) 1111 (3) 10110 (4) 110111

1.4.5 试用十六进制数写出下列字符的 ASCII 码的表示:

- (1) + (2) @ (3) you (4) 43

1.5 二值逻辑变量与基本逻辑运算

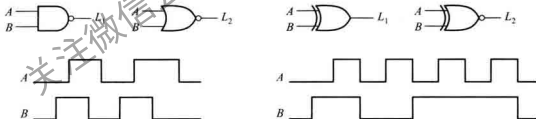
1.5.1 在图题 1.5.1 中, 已知输入信号 A 、 B 的波形, 画出各逻辑门输出的波形。



图题 1.5.1

1.5.2 在图题 1.5.2 中, 已知输入信号 A 、 B 的波形, 画出各逻辑门输出的波形。

1.5.3 在图题 1.5.3 中, 已知输入信号 A 、 B 的波形, 画出各逻辑门输出的波形。



图题 1.5.2

图题 1.5.3

1.6 逻辑函数及其表示方法

1.6.1 已知逻辑函数的真值表如表题 1.6.1 所示, 试写出其逻辑函数表达式。

表题 1.6.1 题 1.6.1 的真值表

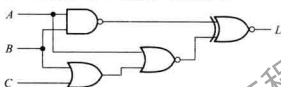
A	B	C	L
0	0	0	1
0	0	1	0
0	1	0	0

续表

A	B	C	L
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

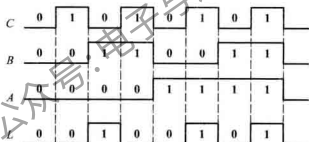
1.6.2 已知逻辑函数 $L = A + \overline{B}C + \overline{A}B\overline{C}$, 试求其对应的真值表。

1.6.3 已知逻辑图如图题 1.6.3 所示, 试求 L 的逻辑函数表达式。



图题 1.6.3

1.6.4 已知逻辑函数 L 的波形图如图题 1.6.4 所示, 试求其真值表和逻辑函数表达式。



图题 1.6.4